ADM Project Report

`Student` : **Azizbek Gulomov (947597)**
`Course` : Advanced Data Danagement
`Instructor` **:** Prof. Andrea Campagner
`Date` :

`Database technologies used` **:**

- **Relational DBMS (PosgreSQL)**
- **Document DBMS (MongoDB)**
- **Graph DBMS (Neo4J)**
- **Distributed Processing**: Apache Spark (PySpark, GraphFrames)

At the end of the report, there is a performance comparison between different database technologies across various data configurations.

`Data configurations used for benchmarking` :

- Users: 1k, 10k, 50k
- Trips: 10k, 50k, 100k
- Events per trip: 0, 2, 5, 10

`Github repository` **:** https://github.com/AzizbekGulomov2002/adm-2026-project

At the end of report, there is comparison performance between different databases for various type of queries on various possible combination of data. We have used following combination:

- Users: 1k, 10k, 50k
- Trips: 10k, 50k, 100k
- Events per trip: 0, 2, 5, 10

PART 1 - Comparing the Relational and Document models

1.1.1 Data Modeling

Relational schema - **PostgreSQL** (`part1/setup_postgres.py)`

- Four tables are defined: users, stations, trips, events. All relationships are expressed through foreign keys. Events are stored in a separate table, referencing trips by trip_id.
- **Tables**:
  - users (id,name,surname,birthdate,country)
  - stations (id,name,city,capacity)
  - trips (id, user_id → users, start_station_id → stations, end_station_id → stations, start_time, end_time, cost)
  - events (id, trip_id → trips, timestamp, type, value)
- **Indexes created:**
  - idx_trips_user, idx_trips_start, idx_trips_end
  - idx_events_trip, idx_events_type

- **Design decision - REFERENCING:**
  - All related data is stored in seperate tables and linked via **foreign keys**. This is the standart normalized approach.
  - **Advantages**: no data dublication, simple updates, strong integrity.
  - **Disadvantages**: multi-table JOINs required for most queries.

- Document schema - **MongoDB** (`part1/setup_mongodb.py`)
  - Three collections: users, stations, trips
  - Events are EMBEDDED inside each trip document.

- **Collection structure**:

```
users: { id,name,surname,birthdate,country }
stations: { id,name,city,capacity }
trips:{ id,user_id,start_station_id,end_station_id,
          start_time, end_time, cost,
          events: [{type,timestamp,value}, ....]}
```

  - **Design decision** - EMBEDDING events inside trips:
    - Events always belong to one trip and are never queried independently. Embedding eliminates the need for $lookup on events, making Q4 (filter by ERROR type) significantly faster.
    - users and stations remain REFERENCED by id inside trips, because the same user/station can appear in many trips and embedding would cause massive duplication.

  - `Comparision summary:`

Aspect	PosgreSQL	MongoDB
Events storage	Seperate table	Embedded in trip doc
Q4 (Error filter)	JOIN events + trips	Direct array filter
Schema change	ALTER TABLE needed	No migration needed
Q2 (aggregation)	Native GROUP BY	$group pipeline
Data consistency	ACID enforced	Eventual (default)
Query language	SQL	Aggregation pipeline

1.1.2 Query Implementation

Scripts: part1/queries_postgres.py | part1/queries_mongodb.py
Benchmark: part1/benchmark.py

- **Q1: All trips with user info and start/end station names**
- `PostgreSQL:`

```sql
SELECT t.id, u.name, u.surname, u.country,
             s1.name AS start_station, s2.name AS end_station,
             t.start_time, t.end_time, t.cost

FROM trips t
JOIN users u ON t.user_id = u.id
JOIN stations s1 ON t.start_station_id = s1.id
JOIN stations s2 ON t.end_station_id = s2.id

# Uses 3-way JOIN with indexed foreign keys
# Scales well up to 100K trips (0.19 seconds)
```

- `MongoDB:`

```
db.trips.aggregate([
        { $lookup: from users, localField: user_id .... },
        { $lookup: from stations, localField: start_station_id .... },
        { $lookup: from stations, localField: end_station_id .... },
        { $unwind + $project }
        )]

# 3 x $lookup required (equivalent of 3 JOINs). Significantly slower
# than Postgres at scale:
    # 908 seconds at u=50K, t=100K because $lookup is not as optimized
    # as native SQL JOIN. This is the main bottleneck of the document
    # model for Q1.
```

- **Q2: All users with trip count and average duration**
- `PostgreSQL:`

```
SELECT u.id, u.name, COUNT(t.id) AS trip_count,
                AVG(EXTRACT(EPOCH FROM (t.end_time - t.start_time)) / 60)
FROM users u LEFT JOIN trips t ON u.id = t.user_id
GROUP BY u.id, u.name, u.surname
```

- `MongoDb:`

```
db.trips.aggregate([
            { $addFields: { dur: {$subtract: [end_time, start_time] } } },
            { $group:{_id: user_id, trip_count: $sum:1, avg_ms: $avg:dur }}
            )]

# Then merged with users cursor in Python
# Both perform similarly. MongoDB's $group pipeline is efficient for
# aggregation without needing $lookup.
```

- **Q3: All stations with trips starting/ending there**
- `PostgreSQL:`

```
SELECT
    s.id,
    s.name,
    s.city,
    COUNT(DISTINCT t_start.id) AS trips_starting,
    COUNT(DISTINCT t_end.id)   AS trips_ending
FROM stations s
LEFT JOIN trips t_start ON s.id = t_start.start_station_id
LEFT JOIN trips t_end   ON s.id = t_end.end_station_id
GROUP BY s.id, s.name, s.city
ORDER BY s.name;

-- BOTTLENECK: At 100K trips, this takes ~18.19s because
-- both LEFT JOINs perform full table scans on trips.
```

- `MongoDB:`

```
db.trips.aggregate([
        { $addFields: {dur: {$subtract: [end_time,start_time] } } },
        { $group: {_id: user_id, trip_count: $sum:1, avg_ms: $avg:dur } }
        )]
# Then merged with users cursor in Python
# Both perform similarly. MongoDB's $group pipeline is efficient for
# aggregation without needing $lookup
```

- **Q3: All stations with trips starting/ending there**
- `PostgreSQL:`
  - Two LEFT JOINs on **trips.start_station_id** and **end_station_id** , grouped by station.
  - BOTTLENECK: At 100K trips, PostgreSQL **Q3** takes **18.19** seconds.
    This is because both JOINs scan the full trips table. A composite index or **MATERIALIZED VIEW** would help.
- `MongoDB:`
  - Two seperate $group pipelines (one per direction), then merged in Python. Very fast: **0.11seconds** at **100K** trips. Q3 is the strongest advantage of the document model.

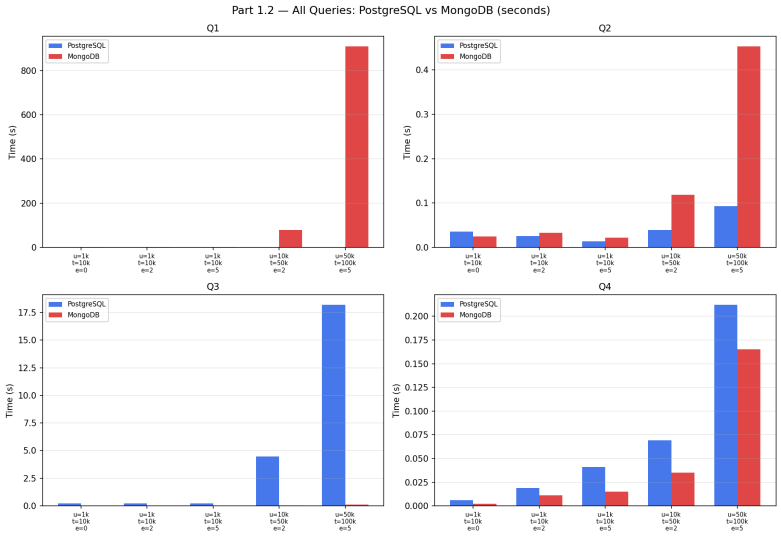
- **Q4: Trips with at least one ERROR event**
- `PostgreSQL:`

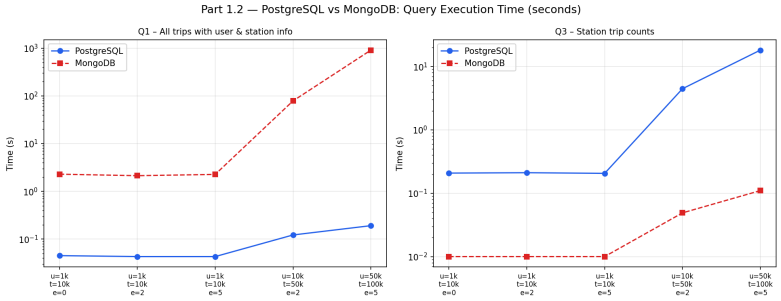
```
SELECT DISTINCT t.id ...
FROM trips t JOIN events e ON t.id = e.trip_id
WHERE e.type = 'ERROR'

# Required JOIN + filter + DISTINCT
# Grows with number of events per trip.
```

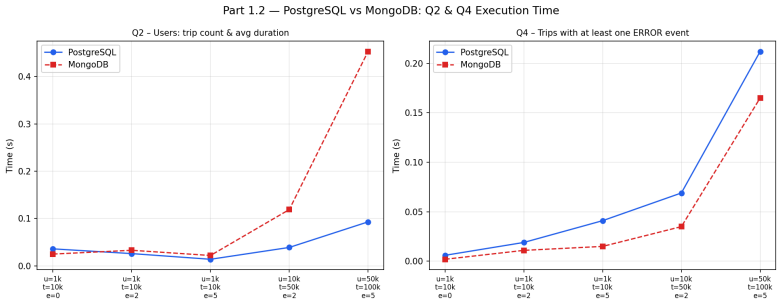
- `MongoDB:`

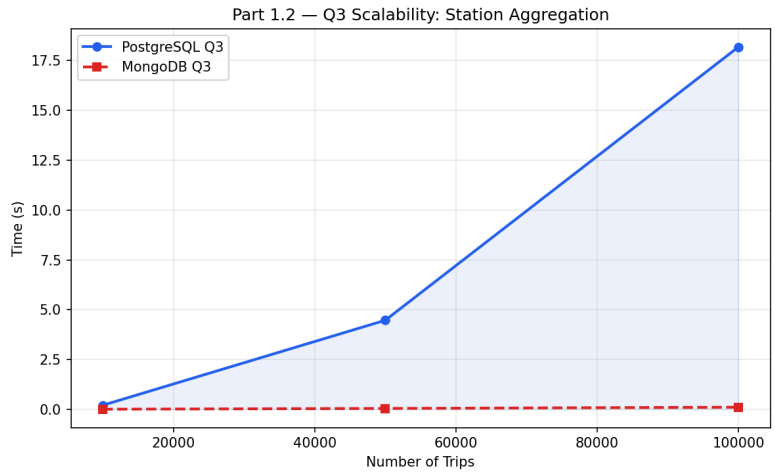
```
db.trips.find({ "events.type":"ERROR"})

# Direct array field filter - no JOIN needed.
# Consistently fastest query in MongoDB.
# This is the key benefit of embedding events.
```

Config	PG-Q1	PG-Q2	PG-Q3	PG-Q4	MG-Q1	MG-Q2	MG-Q3	MG-Q4
u=1k, t=10k, e=0	0.045	0.036	0.208	0.006	2.289	0.025	0.010	0.002
u=1k, t=10k, e=2	0.043	0.026	0.211	0.019	2.128	0.033	0.010	0.011
u=1k, t=10k, e=5	0.043	0.014	0.206	0.041	2.264	0.022	0.010	0.015
u=10k, t=50k, e=2	0.122	0.039	4.470	0.069	78.518	0.119	0.049	0.035
u=50k, t=100k, e=5	0.191	0.093	18.180	0.212	908.626	0.453	0.110	0.165


Part 1.2 — All Queries: PostgreSQL vs MongoDB (seconds)
Q1
Q2
Q3
Q4
PostgreSQL
MongoDB
Time (s)
u=1k t=10k e=0
u=1k t=10k e=2
u=1k t=10k e=5
u=10k t=50k e=2
u=50k t=100k e=5


Part 1.2 — PostgreSQL vs MongoDB: Query Execution Time (seconds)
Q1 – All trips with user & station info
Q3 – Station trip counts
PostgreSQL
MongoDB
Time (s)
u=1k t=10k e=0
u=1k t=10k e=2
u=1k t=10k e=5
u=10k t=50k e=2
u=50k t=100k e=5


Part 1.2 — PostgreSQL vs MongoDB: Q2 & Q4 Execution Time
Q2 – Users: trip count & avg duration
Q4 – Trips with at least one ERROR event
PostgreSQL
MongoDB
Time (s)
u=1k t=10k e=0
u=1k t=10k e=2
u=1k t=10k e=5
u=10k t=50k e=2
u=50k t=100k e=5



- `Key observations:`
  - **PostgreSQL:**
    - Q1, Q2, Q4 scale well - all under **0.22seconds** at maximum config
    - Q3 is the only bottleneck: 18.18 seconds at 100K trips due to double LEFT JOIN aggregation on large table
    - Consistent and predictable performance across event counts
  - **MongoDB:**
    - Q1 is the critical weakness: 908 seconds at 50K users / 100K trips because $lookup (aggregation JOIN) is much slower than native relational JOIN when datasets are large
    - Q3 and Q4 are the strengths: 0.11 seconds and 0.165 seconds at max config because events are embedded (no cross-collection join)
    - Q3 benefits from two seperate $group passes without expensive cross-collection operations
  - **Performance Bottlenecks:**
    - MongoDB Q1 degrades super-linearly with data size due to $lookup
    - PostgreSQL Q3 degreades due to two full-table JOIN+GROUP operations
    - Q4: MongoDB wins at all scales (embedded events → direct filter)
    - Q2: Both databases perform similarly (pure aggregation)

1.1.3 Schema evolution

Requirement: Add field "battery_level" (integer, 0-100) to all events of type BATTERY.

`PostgreSQL` - how to handle:

```
ALTER TABLE events ADD COLUMN battery_lebel INTEGER
            CHECK (battery_level BETWEEN 0 and 100);

# This adds the column to ALL event rows, not just BATTERY ones.
# Existing rows get NULL for battery_level. To enforce the rule
# only for BATTERY events, a CHECK constraint referencing the type
# column must be added:

ALTER TABLE events ADD CONSTRAINT chk_battery
        CHECK (type != 'BATTERY' OR battery_level IS NOT NULL);

# Problems:
```

```
# 1. DDL migration required - locks the table during ALTER
# 2. All existing BATTERY rows must be backfilled with a value
# 3. In production, this requires a maintenance window or a multi-step
# migration (add nullable -> backfill -> add NOT NULL)
# 4. Non-BATTERY rows carry an empty better_level column(wasted space)
```

`MongoDB` - how to handle:

```
# No schema migration needed at all.
# Simply insert new BATTERY events with te field

{ type: "BATTERY", timestamp: ... , value: ...., battery_level: 75 }

# Old BATTERY events without battery_level remain valid documents.
# Application code checks for field existence when needed:

{ db.trips.find({ "events.type":"BATTERY",
                  "events.battery_level":{$exists: true}})
```

- **Conclusion -** Evolvability**:**
  - **MongoDB** is clearly superior for this type of schema evolution. The document model allows adding optional fields to a subset of embedded documents without any migration, table locking, or backfill operations
  - **PosgteSQL** requires DDL changes that affect the entire table and can disrupt production workloads.
  - **Winner:** MongoDB (schema-on-read flexibility)

1.1.4 Spark-based Implementation - Quey 2

Script: part1/spark_query2.py
Core logic:

```
trips = spark.read.csv("data/trips.csv",header=True,inferSchema=True)
users = spark.read.csv("data/users.csv",header=True,inferSchema=True)

trips2 = trips.withColumn("dur_min",
 (F.unix_timestamp("end_time") - F.unix_timestamp("start_time")) / 60)

agg = trips2.groupBy("user_id").agg(
    F.cound("id").alias("trip_count"),
    F.round(F.avg("dur_min"), 2).alias("avg"duration_min")
)

result = users.join(agg,users.id==agg.user_id, "left").drop("user_id")

# Key steps:
   # 1. Load users and trips CSVs as Spark DataFrames
   # 2. Compute duration column in minutes using unix_timestamp
   # 3. Group by user_id: count trips and average duration
   # 4. Left join with users to include users with zero trips


# Results (u=50K, t=100K, e=5)
    # Spark Q2: 0.181 seconds for 50.000 rows
    # PostgreSQL Q2: 0.093 seconds
    # MongoDB Q2: 0.452 seconds
```

- `Comparision:`

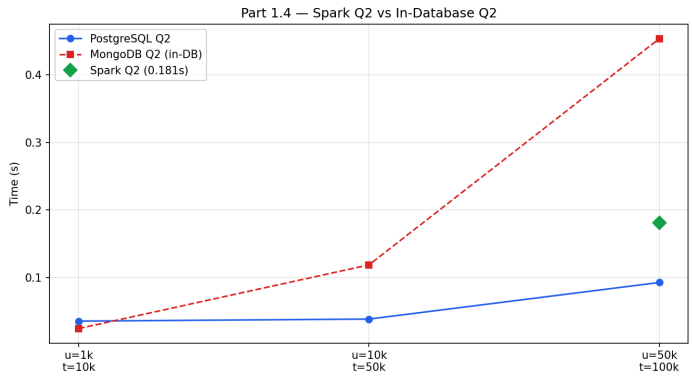
Method	Time	Notes

PostgreSQL Q2	0.093 s	Fastest - native GROUP BY + index
Spark Q2	0.181 s	~2x slower, but no DB required
MongoDB Q2	0.453 s	Slower due to Python-side merge

Spark is slower than PostgreSQL for this query because:

- Spark runs in local mode (no cluster parallelism benefit)
- JVM startup and task scheduling add overhead
- For 50k users / 100k trips, data fits easily in memory

Spark advantage: works on CSV files directly without any database setup, and scales horizontally on a real cluster. At larger scales (millions of rows), Spark would outperform both single-node PostgreSQL and MongoDB.



PART 2 - GRAPH MODEL

2.2.1 Graph Query Implementation - Neo4j

Script: part2/setup_neo4j.py | part2/queries_neo4j.py
Graph model:

```
Node types:
    (:USER)    — id, name, surname, country
    (:STATION) — id, name, city
    (:TRIP)    — id, cost

  Edge types:
    (:USER)-[:PERFORMED]->(:TRIP)
    (:TRIP)-[:STARTS_AT]->(:STATION)
    (:TRIP)-[:ENDS_AT]->(:STATION)


# This structure allows natural traversal:
# user -> trips -> stations (both start and end)
```

- **Q1: All stations reachable by a given user**
- **Core logic:**

```
MATCH (u:USER {id: $uid})-[:PERFORMED]->(t:TRIP)
        -[:STARTS_AT|ENDS_AT]->(st:STATION)
```

```
  RETURN DISTINCT st.id, st.name, st.city
  ORDER BY st.name

# Traversal: USER → TRIP → STATION (2 hops, both directions)
# DISTINCT removes duplicate stations visited via multiple trips.
# This is a simple 2-hop graph traversal — extremely fast in Neo4j.
```

- **Q2: Top 3 most important stations by total trips**
- **Core logic:**

```
MATCH (t:TRIP)-[:STARTS_AT|ENDS_AT]->(st:STATION)
  WITH st, count(t) AS total
  ORDER BY total DESC LIMIT 3
  RETURN st.id, st.name, total

# Counts all STARTS_AT + ENDS_AT edges pointing to each station.
# This is equivalent to incoming + outgoing trip degree
```

  - **Performance discussion:**
    Both queries run in milliseconds at small scale (1k users / 10k trips).
    Neo4j uses index-backed node lookup for {id: $uid} in Q1.
    Q2 traverses all **TRIP→STATION** edges, which grows linearly
    with number of trips — still fast due to pointer-based storage.
  - At larger scale (**50k users / 100k trips**), Neo4j handles graph
    traversal much more efficiently than SQL JOINs or MongoDB $lookup
    because relationships are stored as direct pointers (no index scan).

2.2.2 Spark GraphFrames Implelementation

- **Script:** part2/spark_graphframes.py
- **Vertices:** stations (id,name,city)
- **Edges:** trips projected as (start_station_id → end_station_id)

```
vertices = stations.selectExpr("CAST(id AS STRING) AS id", ...)
  edges    = trips.selectExpr(
      "CAST(start_station_id AS STRING) AS src",
      "CAST(end_station_id   AS STRING) AS dst")
  g = GraphFrame(vertices, edges)
```

- **Q1: (Part 2.2): PageRank - Top3 stations**

```
pr = g.pageRank(resetProbability=0.15, maxIter=5)
  pr.vertices.orderBy("pagerank", ascending=False).limit(3).show()
```

Result: (u=50K, t=100K, e=5)

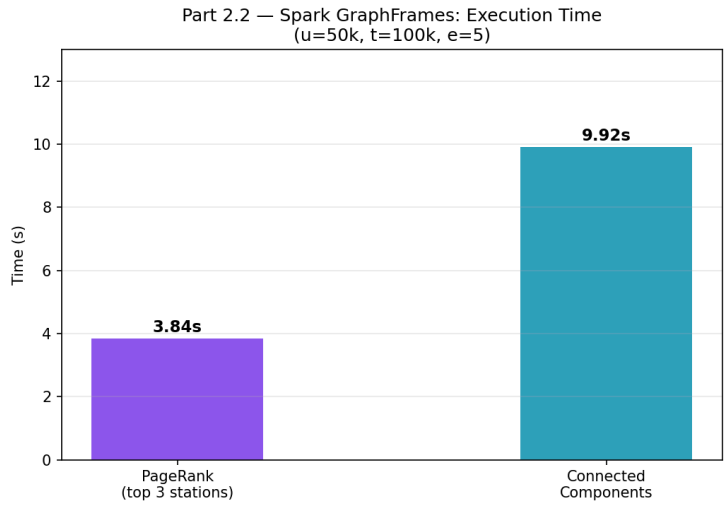
Rank	Station	City	PageRank
1	Station_8	Florence	1.039
2	Station_37	Florence	1.038
3	Station_43	Rome	1.028

  - **PageRank** identifies stations that are not only frequently visited,
    but are reached from other important stations.
    All three top stations are in **Florence and Rome,** consistent with
    high-traffic urban mobility patterns in the simulated data.

- **Q2: (Part 2.2): Connected components of Station subgraph**

```
spark.sparkContext.setCheckpointDir("/tmp/spark_ckpt")
  cc = g.connectedComponents()
  cc.groupBy("component").count().show()
```

  - **Result:**
    component | count
    25769803776 | 50
    Time: **9.92s**
  - All 50 stations belong to a single connected component. This means every station is reachable from every other station through trip paths (directly or indirectly). In a real deployment, isolated stations would appear as separate components — useful for detecting underused stations.



  - **Performance discussion:**
    PageRank (3.84s) is faster than Connected Components **(9.92s).**
    CC requires iterative message passing with checkpointing, which adds I/O overhead. Both run on **50 station** vertices with **100k** trip-derived edges. On a real cluster, both algorithms would scale near-linearly with more executors.

PART3 - Partitioning and Replication

2.3.1 - Partitioning

- **Two strategies considered for the trips collection:**
  - A) Partition by user_id
  - B) Partition by start_station_id

Strategy A - Partition by user_id:

All trips of the same user live on the same shard.
Benefits:

- Q1 (trips with user info): all trip data for a user is local → no cross-shard scatter needed if user filter applied
- Q2 (trip count + avg duration per user): all trips per user are co-located → single-shard GROUP BY per user → fast

Neo4j Q1 (reachable stations for user): all user trips local

Drawbacks:

- Q3 (station aggregations): trips starting at the same station are spread across all user shards → full scatter-gather required to count starts/ends per station → slow
  Neo4j Q2 (top stations): same problem as Q3

Strategy B - Partition by start_station_id:

All trips starting at the same station live on the same shard.
Benefits:

- Q3 (starts per station): trips_starting count is local → fast
  Neo4j Q2 (important stations): starting-side data is local

Drawbacks:

- Q3 still needs scatter-gather for trips_ending count (end_station_id is on a different shard)
- Q1: a user's trips are scattered → cross-shard JOIN needed
- Q2: user aggregation requires full scatter-gather

Decision - Recommended: Partition by user_id

- **Rationale:**
  The most frequent queries in this system are user-centric
  (Q1: trip history per user, Q2: user statistics).
  User-based partitioning ensures these run efficiently.
  Q3 is a less frequent administrative/analytics query;
  its scatter-gather cost is acceptable.
  In MongoDB, sharding by { user_id: "hashed" } distributes
  users evenly while keeping user trips co-located.

2.3.2 Replication

- **Setup: single-leader async replication:**
  - 1 primary + 2 secondary replicas. Async replication means: writes are acknowledged by the primary before being propagated to secondaries. Secondaries may lag behind by milliseconds to seconds (replication lag).
  - If a read query is served by a **SECONDARY** replica:

    Q1 — All trips with user info and station names: Risk: A recently completed trip may not yet be on the secondary. The query returns stale results — missing the latest trip. Inconsistency type: **READ-YOUR-WRITES** violation. Severity: Medium — user may not see their own last trip.

    Q2 — User trip count and average duration: Risk: trip_count and avg_duration may be undercount if the secondary has not yet received the latest trips. Inconsistency type: **MONOTONIC READ** violation possible (two reads could return different counts). Severity: Medium — analytics results are temporarily wrong.

    Q3 — Station trip counts: Risk: Same as Q2 — counts may be slightly stale. For analytics/reporting purposes, minor lag is acceptable. Severity: Low — approximate counts are usually fine.

    Q4 — Trips with ERROR events: Risk: A trip that just had an ERROR event inserted may not appear on the secondary yet. Severity: High — could cause failure to detect active errors. In a real mobility platform, **ERROR** detection queries should always be routed to the **PRIMARY** replica.

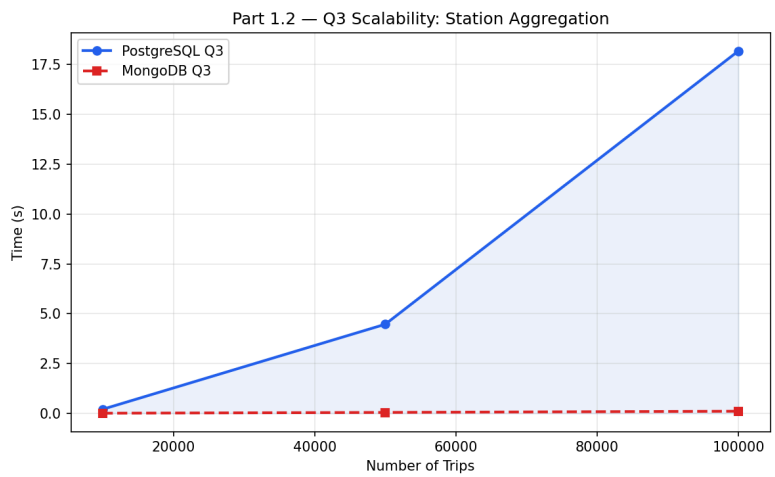
    Neo4j Q1 and Q2: Same considerations apply — new trips and their edges may not yet be replicated to secondaries. Graph traversal on stale data returns incomplete paths.
  - **Recommendation:**
    Route real-time / safety-critical queries (Q4, Neo4j Q1)
    to the PRIMARY replica.
    Route analytics queries (Q2, Q3, Neo4j Q2) to secondaries
    to distribute read load — small lag is acceptable.

Use read preferences: primaryPreferred for Q1 and Q4, secondaryPreferred for Q2 and Q3.

2.3.2 Performance comparision report

- We benchmarked 4 queries across 5 data configurations:

Config	PG-Q1	PG-Q2	PG-Q3	PG-Q4	MG-Q1	MG-Q2	MG-Q3	MG-Q4
u=1k, t=10k, e=0	0.045	0.036	0.208	0.006	2.289	0.025	0.010	0.002
u=1k, t=10k, e=2	0.043	0.026	0.211	0.019	2.128	0.033	0.010	0.011
u=1k, t=10k, e=5	0.043	0.014	0.206	0.041	2.264	0.022	0.010	0.015
u=10k, t=50k, e=2	0.122	0.039	4.470	0.069	78.518	0.119	0.049	0.035
u=50k, t=100k, e=5	0.191	0.093	18.180	0.212	908.626	0.453	0.110	0.165



- **PostgreSQL Observations:** Q1, Q2, Q4 remain fast at all scales (under 0.22s). Q3 is the bottleneck: 18.18s at 100k trips. Performance is predictable and consistent. Ideal for join-heavy, user-centric queries.
- **MongoDB Observations:** Q1 collapses at scale: 908s at 50k users / 100k trips. **$lookup** is not a native JOIN — it is implemented as a pipeline stage that iterates and matches per document. Q3 and Q4 are consistently fast due to embedded events. Q2 is efficient via $group aggregation pipeline.
- **Performance Bottlenecks:**
  - MongoDB Q1: $lookup degrades super-linearly. Root cause: no hash-join optimization for cross-collection lookups.
  - PostgreSQL Q3: double GROUP BY on large trips table. Addressable with a MATERIALIZED VIEW or pre-aggregation.
- **Part 1.4 — Spark vs In-Database Q2:** PostgreSQL: **0.093s** | Spark: **0.181s** | MongoDB: **0.453s** Spark is **~2x** slower in local mode due to JVM overhead.
  On a real cluster with 10+ nodes, Spark would scale best.
- **Part 2.2 — GraphFrames:** PageRank: 3.84s | Connected Components: **9.92s**. CC is slower due to iterative algorithm + checkpointing. Both run acceptably for **50 stations** / **100k-edge** graph

CONCLUSION

```
1. For join-heavy trip queries (Q1, Q2): PostgreSQL is significantly faster and scales well. Use PostgreSQL for user-centric dashboards and reports.

2. For event-filtering queries (Q4): MongoDB wins at all scales due to embedded events. Embedding events is the correct design choice for Q4.

3. For station aggregation (Q3): MongoDB is faster due to two independent $group passes without cross-collection joins. PostgreSQL Q3 needs optimization
(e.g., covering index or pre-aggregation).

4. For schema evolution: MongoDB is more flexible — new optional fields can be added to document subsets without any migration.

5. For graph traversal: Neo4j is the best choice. Pointer-based storage makes multi-hop traversal orders of magnitude faster than SQL or document JOIN
approaches.

6. For distributed analytics at scale: Spark (with DataFrames or GraphFrames) is the right tool when data exceeds single-node capacity. PageRank and
Connected Components via GraphFrames are straightforward to implement and scale horizontally.
```